Maintained stable guidance · Package 1.6.0
Documentation f3f33e6a719a7a35d17c70ce449c4e620e685306
Stable version selector · Downloads

Agent Paranoid Android

User Handbook

Reproducible synthetic test data for CSV and database workflows

Contents

Start Here

- Overview
- Installation
- First CSV Dataset
- Related Tables
- Review The Output

How-To Guides

- Add Business Rules
- Profile PostgreSQL
- Profile Through Trino
- Transform Local Data

Integrations

- Connect An MCP Client
- Connect An AI Client
- Containers

Reference

- Reference Index
- Choose An Approach
- Profiles And Specs
- Safety Model

Help

- Troubleshooting
- Known Issues

Start Here

Agent Paranoid Android

Create repeatable test datasets with useful relationships and business rules, without copying source rows. Start with a file, safe database metadata, or an explicit specification, then review the plan before generating fresh values.

Stable `1.6.0` is the recommended release. Use the CLI below or the runnable Python API example.

Five-Minute Check

```
python3 -m pip install "agent-paranoid-android==1.6.0"
test-data-agent doctor
test-data-agent demo --output out/demo
```

The demo uses a bundled fictional fixture. After installation it needs no network, database or AI provider. A healthy installation reports:

```
quickstart smoke: ok
doctor passed
```

Choose Your Starting Point

I have	Start here
One CSV file	First CSV Dataset
Related CSV files	Related Tables
PostgreSQL	PostgreSQL Workflow
Trino	Trino Workflow
A safe profile or a plan	Profiles And Specs

Generation Or Transformation?

Synthetic generation creates new rows from a reviewed `DatasetSpec`. Use it for valid, boundary, load or controlled-invalid test cases. The normal process is profile, review the specification, generate, validate, then review the output.

Selective transformation keeps one-to-one correspondence with input rows and applies explicit field actions. Its result has mixed origin. Source-value preservation requires a non-sensitive field decision and fresh local-terminal approval of the exact inputs and plan; agents and MCP cannot issue it. Start with Transform Local Data.

Rules And Formats

Use business rules for formulas, field conditions, foreign keys and aggregate checks. The source/output matrix shows routes to CSV, JSON, Parquet and generated SQL. SQL export writes a file, not a database.

Advanced Integrations

AI, MCP, Containers, and Custom providers

Before Sharing A Dataset

Review the specification, validation reports and `generation_manifest.json`. For a valid synthetic dataset expect `synthetic: true`, `source_rows_copied: false` and `validation_valid: true`. These checks do not certify statistical anonymity or detect every sensitive value. Explicit mixed/negative datasets can intentionally fail validation.

Database access remains allowlisted, read-only and bounded. AI receives safe metadata, never raw PII, secrets or source rows. See the Safety Model and Troubleshooting.

For offline reading use Downloads. For another release use Documentation Versions. Maintainer and release records live in History And Maintenance.

Installation

Stable `1.6.0` is the recommended release. Its signed source, packages and containers passed public verification.

Requirements

- Python 3.11 or newer
- enough local disk space for the requested output
- a dedicated working directory for inputs and generated artifacts

The supported CI matrix covers CPython 3.11, 3.12, 3.13, and 3.14. Python 3.11 remains the minimum so users can adopt newer interpreters without forcing existing environments to upgrade. The notice and release-gate rules for changing this matrix are defined in Runtime And Integration Support.

Read the migration notes before regenerating existing fixtures.

Install From PyPI

The stable release is `1.6.0`; the commands below pin that exact version for reproducible installation.

Create an isolated environment:

macOS and Linux

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install --upgrade pip
python3 -m pip install "agent-paranoid-android==1.6.0"
```

The base package supports CSV, JSON, and deterministic PostgreSQL SQL export. Add only the source and format integrations you need:

```
python3 -m pip install "agent-paranoid-android[parquet]==1.6.0"
python3 -m pip install "agent-paranoid-android[mcp]==1.6.0"
python3 -m pip install "agent-paranoid-android[mcp,trino]==1.6.0"
python3 -m pip install "agent-paranoid-android[postgres]==1.6.0"
python3 -m pip install "agent-paranoid-android[openai]==1.6.0"
python3 -m pip install "agent-paranoid-android[gigachat]==1.6.0"
```

Dependency footprint budgets live in Dependency Compatibility.

The `all` extra remains available for development, demos, and container builds. It is not the recommended user installation. Use `test-data-agent doctor --require-extra all` to verify that full environment.

The experimental GigaChat adapter is retained in `1.6.0` through its explicit `gigachat` extra. Follow Use The GigaChat Advisor for authentication, mandatory TLS verification, and the review-first workflow. The default provider remains OpenAI.

Stable `1.6.0` is the recommended default. Add an optional extra to the same exact stable pin only when needed.

Windows PowerShell

```
py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install "agent-paranoid-android==1.6.0"
```

Run the self-contained environment check:

```
test-data-agent doctor
```

Expected final lines:

```
quickstart smoke: ok
doctor passed
```

`doctor` uses a temporary fictional fixture and makes no external calls. Use `doctor --require-extra parquet` (or another installed extra) for an offline capability check. See Troubleshooting for individual checks and CLI automation for JSON output.

For development setup use Contributing. For documentation builds use Publishing Documentation. Shell completion has its own reference.

Confirm The Installed Version

```
python3 -c "import test_data_agent; print(test_data_agent.__version__)"
```

The package name is `agent-paranoid-android`. The command remains `test-data-agent`.

Next Step

Use First CSV Dataset for one table, Related Tables for a folder containing one CSV per table, or Profile PostgreSQL for an allowlisted database.

First CSV Dataset

This tutorial profiles one CSV file, infers a generation specification, generates new values, and validates the result.

Prerequisites

- complete Installation;
- run commands from a clone of this repository so the synthetic fixture below is available;
- choose a new output directory.

The fixture contains fictional values and reserved example email domains.

For the same workflow as an executable script with its own synthetic fixture, run `examples/csv_quickstart/run.sh`.

Generate 25 Rows

```
test-data-agent generate-from-csv tests/fixtures/customers.csv \
  --count 25 \
  --seed 12345 \
  --format csv \
  --output out/customers.csv
```

Expected summary:

```
Generated synthetic dataset: out | rows: customers=25 | seed: 12345 | validation: passed
| source rows copied: no
```

The exact output path shown in the summary can vary by platform.

Inspect The Bundle

The command writes:

```
out/
  customers.csv
  csv_profile.json
  dataset_spec.json
  generation_manifest.json
  validation_report.json
```

Check the manifest before using the data:

```
python3 -c "import json; m=json.load(open('out/generation_manifest.json'));
```

```
print(m['synthetic'], m['source_rows_copied'], m['validation_valid'], m['seed'])"
```

Expected output:

```
True False True 12345
```

Open `out/customers.csv` and confirm that it contains 25 generated rows plus the header. Source rows are not shuffled or copied.

Reproduce The Result

Generate into a different path with the same seed:

```
test-data-agent generate-from-csv tests/fixtures/customers.csv \
  --count 25 \
  --seed 12345 \
  --format csv \
  --output out/repeated/customers.csv
```

The two generated CSV files should match:

```
python3 -c "from pathlib import Path; assert Path('out/customers.csv').read_bytes() ==
Path('out/repeated/customers.csv').read_bytes(); print('deterministic: ok')"
```

Use Your Own CSV

Replace the fixture path with your file:

```
test-data-agent generate-from-csv data/customers.csv \
  --count 100 \
  --seed 12345 \
  --format csv \
  --output out/customers.csv
```

Treat every source CSV as potentially sensitive. Do not share its contents, profile artifacts, or command logs until you have reviewed them. The profiler suppresses raw values for fields detected as sensitive, but ambiguous domain fields still require human review.

Exact values remain replaced by default. When `customers.status` has been reviewed as a bounded non-sensitive business enum, authorize only that field:

```
test-data-agent generate-from-csv tests/fixtures/customers.csv \
  --local-category customers.status \
  --count 25 \
  --seed 12345 \
```

```
  --format csv \
  --output out/approved-status/customers.csv
```

The request fails closed if the field contains PII, secrets, identifiers, quasi-identifiers, free text, excessive cardinality, or long values. The option does not authorize any other field or any external-provider destination.

Review ambiguous identifiers and financial columns before accepting inferred keys or distributions. For exact category limits, distinct-count evidence and identifier behavior, see CSV Profiling Details.

Next, read Review The Output.

Related Tables

Use this workflow when a folder contains one CSV file per related table. The file stem becomes the entity name.

Input Layout

```
example_dataset/
  customers.csv
  orders.csv
```

The checked-in fixture already has this shape.

For an executable version with synthetic inputs and business rules, run `examples/relational_csv/run.sh`.

Generate The Dataset

```
test-data-agent generate-from-example tests/fixtures/example_dataset \
  --count 25 \
  --seed 12345 \
  --format csv \
  --output out/example_dataset
```

Expected summary:

```
Generated synthetic dataset: out/example_dataset | rows: customers=25, orders=25 | seed:
12345 | validation: passed | source rows copied: no
```

The output contains:

```
out/example_dataset/
  customers.csv
  orders.csv
  profile.json
  dataset_spec.yaml
  generation_manifest.json
  validation_report.json
```

Review The Relationship

Open `out/example_dataset/dataset_spec.yaml` and locate `relationships`. The inferred customer/order relationship should name:

```
parent_entity: customers
parent_field: customer_id
```

```
child_entity: orders
child_field: customer_id
```

The generator creates new parent identifiers first and assigns child foreign keys from those generated values. Source identifiers are not reused.

Run The Stages Separately

Use separate commands when the inferred specification needs review or editing:

```
test-data-agent profile-example tests/fixtures/example_dataset \
  --output out/profile.json

test-data-agent infer-spec out/profile.json \
  --count 25 \
  --output out/dataset_spec.yaml

test-data-agent generate out/dataset_spec.yaml \
  --seed 12345 \
  --format csv \
  --output out/generated

test-data-agent validate out/dataset_spec.yaml out/generated \
  --output out/generated/validation_report.json
```

Do not generate until a reviewer accepts inferred relationships and constraints for a new domain.

To preserve one reviewed local enum across the staged workflow, pass the same field-scoped selector while profiling:

```
test-data-agent profile-example tests/fixtures/example_dataset \
  --local-category orders.status \
  --output out/profile-with-status.json
```

`orders.cancellation_reason` is free text and is not eligible. The allowlist does not copy source rows or expose exact values to external advisors or default MCP responses.

See Profiles And Specs for the purpose of each stage.

Review The Output

A successful command is not the end of the review. Use the artifacts in the bundle to decide whether the synthetic dataset is safe and useful.

This page describes synthetic-generation bundles. The gated mixed-origin transformation workflow has a separate review contract: validation does not make its output synthetic, and action provenance and any required exact-snapshot local preservation receipt must also be checked. See Selective Transformation.

Review Order

1. Read `generation_manifest.json`.
2. Read `validation_report.json`.
3. Read `business_validation_report.json` when business rules were supplied.
4. Review the effective `dataset_spec.yaml` or `dataset_spec.json`.
5. Inspect a small sample of generated rows.

Manifest Checks

For a valid synthetic dataset, require all of these conditions:

```
{
  "synthetic": true,
  "source_rows_copied": false,
  "validation_valid": true
}
```

Also confirm:

- `seed` is the value requested by the caller;
- `row_counts` match the requested scale;
- `output_format` is correct;
- `package_version` and `dataset_spec_schema_version` are recorded;
- `spec_sha256` is present;
- `business_validation.rules_sha256` and its summary are present when rules were used.
- `effective_rules` binds generation mode, validation settings, relationship, constraint, and distribution counts to the exact spec and business-rule fingerprints without repeating rule values.

Validation Report

`validation_report.json` reports deterministic checks for:

- entity and field schema;
- primary-key uniqueness;
- relationships and foreign keys;
- field and row constraints;
- temporal and formula constraints;
- aggregate reconciliation.

Do not treat a missing report as success. A failed valid-mode run is not an accepted dataset. Explicit mixed/negative test generation may retain intentionally invalid rows and their failed validation report in a supported output format; the CLI reports `validation_failed` and exit 1. Do not relabel that result valid. Parquet negative datasets use explicitly marked scalar columns when malformed cells cannot fit native Arrow types. The bundle reader restores those cells for validation; their report remains invalid. Ordinary incompatible typed values still reject the entire export without partial publication. Keep the specification, manifest and encoding metadata described in the source/output matrix. Privacy failures never authorize publication of source values.

These synthetic manifest checks do not apply to the separately gated selective transformation workflow. Its bundle records mixed origin, action provenance and any exact-input preservation authority; it must not claim fully synthetic or anonymous output. Transformation validation failure prevents publication. See the execution contract.

Assurance Levels

Interpret each result at its actual level:

Level	Evidence	What it does not prove
Structural validation	Schema, keys, relationships, constraints, formulas, and business rules passed deterministic checks.	Values are private or safe to distribute.
Built-in safety policy	The reviewed spec rejected known raw sensitive values, unsafe categories, and disabled safety settings. The report's `privacy` section records this policy check.	Detectors found every PII, secret, quasi-identifier, or rare free-text value.
Complete-row reuse check	A supported CSV workflow compared generated and available source rows and found no complete-row match.	No individual value, partial row, rare combination, or statistical pattern resembles the source.

Level	Evidence	What it does not prove
Domain privacy review	An accountable reviewer assessed the generated artifacts under the organization's policy and intended use.	A formal statistical privacy guarantee unless a separate qualified evaluation provides one.

`validation_valid: true` means the enabled deterministic checks passed. It is not a privacy certificate. Name- and value-based detection is heuristic and can produce false negatives, especially for domain identifiers, quasi-identifiers, rare categories, and free text. This project does not assign an anonymity, differential-privacy, or re-identification-resistance level.

Profile Review

Profiles should contain metadata such as types, null ratios, ranges, percentiles, approximate distinct counts, safe low-cardinality distributions, and masked patterns.

Stop and investigate if a profile contains:

- a real email address, phone number, token, credential, or private key;
- a source row copied as a nested object or list;
- rare free-text values that could identify a person;
- an unexpected identifier distribution.

Generated-Row Review

Inspect generated data for domain usefulness, but do not compare or publish raw source rows during that review. For sensitive domains, perform review in the same protected environment as the source data.

Current CSV bundles declare `csv_encoding: backslash-null-v1` in their manifest. String cells and headers that could be spreadsheet formulas receive a reversible `\F` marker; null is `\N`, empty text stays empty, and literal leading backslashes are doubled. Numeric values, including negative numbers, remain numeric. Keep the manifest for schema-aware bundle decoding; legacy unmarked CSV retains its previous interpretation. Artifact names supplied through public workflows must be one safe filename component; separators and parent-directory components are rejected before any artifact is written. Formula failures expose only fixed local reasons, never expressions, values, or nested exception text.

Re-run with the same spec, rules, package version, and seed when reproducibility is required. A change to any of those inputs can legitimately change output.

See Safety Model for guarantees and limitations.

How-To Guides

Add Business Rules

Use structured YAML or JSON when generated rows must satisfy domain rules. The deterministic rule engine applies and validates the rules; an LLM is not the enforcement mechanism.

For a complete annotated format, see the runnable business-rules reference, including all rule families, condition operators, formula modes, aggregates, scenarios and a matching DatasetSpec. The standalone JSON Schema is generated from the runtime model.

Create A Rule File

The checked-in example at `examples/orders_rules.yaml` contains:

```
field_rules:
  - table: orders
    field: status
    required: true
    allowed_values: [paid, cancelled]
  - table: orders
    field: amount
    required: true
    min_value: 0

row_rules:
  - type: temporal_ordering
    table: orders
    start_field: created_at
    end_field: fulfilled_at
    allow_equal: true

  - type: formula
    table: orders
    field: amount
    expression: quantity * unit_price
```

Rule references must match fields in the effective `DatasetSpec`. Unknown keys, tables, fields, rule types, and formula syntax are rejected.

Prepare A Reviewed Spec

```
test-data-agent profile-example tests/fixtures/example_dataset \
  --output out/rules/profile.json

test-data-agent infer-spec out/rules/profile.json \
  --count 25 \
  --output out/rules/dataset_spec.yaml
```

Review `out/rules/dataset_spec.yaml`, then generate with rules:

```
test-data-agent generate out/rules/dataset_spec.yaml \
  --seed 12345 \
  --format csv \
  --business-rules examples/orders_rules.yaml \
  --output out/rules/generated
```

Review:

```
out/rules/generated/
  business_validation_report.json
  generation_manifest.json
  validation_report.json
```

The generation manifest records a normalized rule fingerprint, rule counts, pass/fail counts, truncation status, overall business validity, and bounded expected-versus-observed violation counts.

Supported Rule Categories

Category	Use
Field rule	Required values, allowed values, numeric bounds
Conditional required	Require fields when a condition matches
Conditional allowed values	Restrict a field under a condition
Temporal ordering	Enforce start/end ordering
Formula	Calculate or validate bounded arithmetic
Foreign key	Preserve cross-table references
Aggregate formula	Validate an aggregate expectation
Scenario	Control weighted combinations of field values

Generate Controlled Invalid Cases

Use an explicit generation mode:

```
test-data-agent generate out/rules/dataset_spec.yaml \
  --count 100 \
  --seed 12345 \
  --format csv \
  --mode mixed \
  --invalid-ratio 0.02 \
  --business-rules examples/orders_rules.yaml \
  --output out/rules/mixed_cases
```

With business rules, `mixed` uses seeded random row selection at the requested invalid ratio and attempts an eligible rule violation for each selected row. The ratio is not an exact count of failed cells or rules. `negative` attempts eligible violations for every row; it does not make every field invalid, and tables without an applicable mutation can remain valid. Keep these datasets clearly separated from valid fixtures and review the validation report rather than expecting it to pass.

For tables with several field and row rules, selected rows are distributed deterministically across required, allowed-value, numeric-bound, conditional, temporal, and formula violations. Reusing the same seed, mode, ratio, and rule file reproduces the same negative cases. Foreign-key violations use synthetic missing parent keys without changing the parent table. Aggregate-formula violations change only the configured concrete numeric field. Count-style aggregate rules with `field: "*"` are validation-only because changing row counts would also break the dataset shape.

One selected invalid mutation can break several overlapping rules. The engine audits the affected row before and after mutation and includes newly broken local rules in the expected counts. Cross-table rules are checked before and after the injection pass; newly broken rules with a valid baseline are also counted. Injection requires a valid business-rule baseline and refuses an already invalid baseline before introducing negative cases. Standalone validation still reports unplanned failures as unexpected. An injection that does not break its selected rule still reports a missing expected failure. These audits share the normal work and deadline budgets.

The business validation report compares expected and observed violations per rule. `unexpected_violation_count` identifies failures that were not selected by controlled invalid generation; `missing_expected_violation_count` identifies selected cases that validation did not observe. `expectations_met` is true only when both counts are zero. These fields contain counts only, never generated row values.

Reproduce The Same Cases Through CLI And MCP

The checked-in `examples/negative_cases/` bundle fixes the spec, rule file, seed (`1300`), mode (`mixed`), and invalid ratio (`0.5`). Run it through CLI:

```
test-data-agent generate examples/negative_cases/dataset_spec.yaml \
  --seed 1300 \
  --mode mixed \
  --invalid-ratio 0.5 \
  --format json \
```

```
  --business-rules examples/negative_cases/business_rules.yaml \
  --output out/negative-cli
```

The MCP equivalent uses the same files after copying the example bundle below the configured generator workspace. The mode and invalid ratio are already recorded in `dataset_spec.yaml`:

```
{
  "name": "generate_dataset",
  "arguments": {
    "spec_path": "negative_cases/dataset_spec.yaml",
    "output_folder": "negative-mcp",
    "output_format": "json",
    "seed": 1300,
    "business_rules_path": "negative_cases/business_rules.yaml"
  }
}
```

Both paths produce identical synthetic row files and matching expected versus observed violation counts. Use separate empty output folders when repeating the example.

Safety Restrictions

Do not put real identifiers, emails, phone numbers, addresses, credentials, tokens, or other production literals into rule files. Sensitive-looking literals are rejected by CLI and MCP entry points.

Formula syntax is a bounded arithmetic subset, not Python or SQL. Rule payload size, expression complexity, and estimated row/rule evaluations are limited.

See Configuration for adjustable limits.

Generation ranges and business invariants

A field's distribution describes sampling. A formula in `assign` mode computes and replaces its target; its result may leave that sampling range. The report records assignment-domain checks and violations separately from business-rule validity. Declare a field rule with `min_value` / `max_value` when the result must obey an invariant. Formula `check` mode validates existing values without assigning them; use it when replacement would hide the condition you intend to test. The complete runnable reference linked above includes both modes and conditional predicates, including null and inclusive range checks.

Scenario weights are relative probabilities, not exact row quotas or predicates that automatically bind a scenario to source rows. A passing rule report checks declared rules; it does not establish source fidelity or discover missing rules. Entity-attribute-value inputs are not automatically pivoted into business fields. Model the desired entities and declare relationships/rules explicitly before using the generation command above.

Profile PostgreSQL

Database profiling is available in stable `1.6.0` and requires explicit read-only configuration, allowlists and resource budgets.

Install the optional driver:

```
pip install "agent-paranoid-android[postgres]"
```

Configure one explicit read-only scope. The password setting names another environment variable; it is not the password itself.

```
export POSTGRES_SOURCE_ID=warehouse
export POSTGRES_HOST=db.example.internal
export POSTGRES_DATABASE=analytics
export POSTGRES_USER=test_data_agent
export POSTGRES_PASSWORD_ENV=WAREHOUSE_PASSWORD
export WAREHOUSE_PASSWORD='replace-in-your-shell'
export POSTGRES_ALLOWED_SCHEMAS=public
export POSTGRES_ALLOWED_TABLES=public.customers,public.orders
export
POSTGRES_ALLOWED_COLUMNS=public.customers.customer_id,public.customers.status,public.ord
ers.order_id,public.orders.customer_id,public.orders.state,public.orders.amount
```

For an intentionally table-wide aggregate profile, an entry may instead use the exact `schema.table.*` form:

```
export POSTGRES_ALLOWED_COLUMNS='public.customers.*,public.orders.*'
```

The parent tables must still appear exactly in `POSTGRES_ALLOWED_TABLES`. Before any aggregate runs, bounded PostgreSQL catalog metadata expands each wildcard into a frozen, sorted explicit-column snapshot. Profiling queries enumerate quoted column identifiers; they never execute `SELECT *`. Bare, schema-wide, table-name, embedded, or over-budget wildcards fail closed.

Declared single-column integer and string primary keys are recognized regardless of their name (for example, `ticket_no` or `record_code`). Generation uses fresh synthetic identifiers; it does not preserve source keys. Sensitive key fields retain their sensitivity classification. Reprofile saved artifacts to obtain this metadata behavior.

Composite primary keys are currently unsupported: profiling rejects them before aggregate scans, including when the column allowlist exposes only part of the key. It never treats one component as a complete unique key. Other primary-key types

require an independently reviewed generation specification. Review the complete relational constraints explicitly when authoring that specification; automatic inference does not support composite uniqueness.

If a platform portal supplies a JDBC endpoint, replace only the separate host, port, database, and TLS settings with a credential-free JDBC-style URL:

```
export POSTGRES_JDBC_URL='jdbc:postgresql://db.example.internal:5432/analytics?
sslmode=verify-full'
```

Keep `POSTGRES_USER`, `POSTGRES_PASSWORD_ENV`, every allowlist, and every budget separate. Userinfo, passwords, unknown query properties, and session-changing options in the URL fail before a connection is opened. This syntax is parsed into the existing Psycopg adapter; Java and JDBC drivers are not used. If both URL and component settings are present, explicitly supplied values must match.

PostgreSQL connections now default to `verify-full`, which verifies the server certificate and configured hostname. Configure a trusted CA certificate through the Psycopg/libpq trust-root configuration (for example `PGSSLROOTCERT`) and use a hostname covered by the certificate. Existing installations using the former `require` default must configure trust before upgrading. Connection failure does not fall back to a weaker mode. Explicit `require`, `verify-ca`, or `disable` requires `POSTGRES_ALLOW_INSECURE=true` and is intended only for isolated local testing; these modes do not provide full server identity verification.

The database role must already be read-only. The client also requests a read-only transaction, TLS, statement and lock timeouts, and bounded aggregate results. It accepts no arbitrary SQL and never profiles source rows.

Table profiling retains observed minimum/maximum bounds for non-sensitive `date` and `timestamp` columns in the existing column-summary query. Inferred generation uses those ranges; timestamp offsets are retained. Sensitive-name columns do not expose date bounds, and all-null columns have none. Invalid or inconsistently typed bounds fail profiling rather than silently inventing a period. Missing bounds elsewhere still require review; this table-profile behavior does not imply date-bound support for `profile-query`.

Create a safe profile, review a generation specification, generate, and validate. Output paths must not already exist unless the command explicitly supports `--overwrite`:

```
test-data-agent profile-postgres --output out/postgres-profile.json
test-data-agent infer-spec out/postgres-profile.json --output out/dataset-spec.yaml
test-data-agent generate out/dataset-spec.yaml --seed 12345 --output out/generated
test-data-agent validate out/dataset-spec.yaml out/generated
test-data-agent export-postgres-sql out/dataset-spec.yaml --seed 12345 --output
out/generated.sql
```

Run the SQL only in the intended local or disposable target database:

```
psql --set ON_ERROR_STOP=1 --dbname synthetic_target --file out/generated.sql
```

The SQL file contains one transaction, deterministic quoted `CREATE TABLE`, foreign-key, and `INSERT` statements, PostgreSQL scalar literals, and `NULL`. It is built from validated generated records, not profile query rows. Render or validation failure leaves no partial output file. Repeating export with the same reviewed spec, seed, package, and recorded environment produces the same logical SQL; compare files directly when verifying one environment:

```
test-data-agent export-postgres-sql out/dataset-spec.yaml \
  --seed 12345 --output out/generated-second.sql
cmp out/generated.sql out/generated-second.sql
```

To profile a reviewed relation without creating a database view, put one fully qualified single-table query in a local file:

```
SELECT o.order_id, o.state, o.amount AS amount
FROM public.orders AS o
WHERE o.order_id < 999999
```

Keep the same physical schema/table/column allowlists and run:

```
test-data-agent profile-query query.sql \
  --adapter postgres \
  --source-id warehouse \
  --entity orders_query \
  --output out/query-profile.json
test-data-agent infer-spec out/query-profile.json \
  --output out/query-spec.yaml
test-data-agent generate out/query-spec.yaml \
  --seed 12345 --output out/query-generated
test-data-agent validate out/query-spec.yaml out/query-generated
```

This generation walkthrough uses direct columns and aliases, all explicitly allowed above. A derived expression such as `amount * 2` can be profiled under the SQL policy, but automatic `infer-spec` rejects profiles with unsupported expression dependencies. Do not infer that the expression is translated into a generation formula; review a separate supported specification instead.

The query is parsed locally and must stay inside the documented scalar, projection, and filter subset. A no-row schema probe and bounded aggregate wrappers execute in the forced read-only session. The profile contains a query fingerprint but not the SQL text, its literal, backend messages, or query rows.

The equivalent typed Python entry point keeps the query path and database configuration explicit:

```
from pathlib import Path

import psycopg

from test_data_agent import (
    SqlQueryAdapter,
    SqlQueryProfileRequest,
    profile_postgres_query_source,
)
from test_data_agent.postgres_config import PostgresConfig

request = SqlQueryProfileRequest(
    adapter=SqlQueryAdapter.POSTGRES,
    source_id="warehouse",
    entity="orders_query",
    query_file=Path("query.sql"),
)
profile = profile_postgres_query_source(
    request,
    config=PostgresConfig.from_env(),
    driver=psycopg,
)
```

Exact values remain disabled by default. A bounded, reviewed, non-sensitive business enum can be kept locally with its source-qualified identity:

```
test-data-agent profile-postgres \
  --local-category warehouse.public.orders.state \
  --output out/postgres-profile.json
```

PII, secrets, identifiers, quasi-identifiers, free text, excessive cardinality, and long values fail closed. Original literals are not sent to providers, MCP, logs, or errors. A column wildcard is not a local-value policy: exact values are considered only when the separate, fully qualified `--local-category` selector is present and its content checks pass.

The selector authorizes only the bounded value domain and aggregate counts. It does not retain source row order or a mapping that reconstructs source rows. The generated SQL may contain approved values because its input is the validated synthetic dataset.

The same profiling boundary is available from Python:

```
import psycopg

from test_data_agent.postgres_client import PostgresClient
from test_data_agent.postgres_config import PostgresConfig
from test_data_agent import generate_dataset, infer_dataset_spec, validate_dataset
from test_data_agent.postgres_profiler import dataset_profile_from_postgres

config = PostgresConfig.from_env()
profile = dataset_profile_from_postgres(PostgresClient(config, psycopg))
spec = infer_dataset_spec(profile)
# Review or edit spec here before generation.
rows = generate_dataset(spec, seed=12345)
report = validate_dataset(rows, spec)
assert report.valid
```

Keep profile and generated artifacts local unless their destination policy explicitly permits otherwise.

For a complete synthetic end-to-end check against a temporary local PostgreSQL cluster, run the `examples/local_postgres` example. It creates a SELECT-only role, proves that writes are denied, profiles two related tables, validates deterministic generation, executes the exported SQL in an empty target database, and removes the cluster. Run `examples/local_postgres/run-jdbc.sh OUTPUT` for the same workflow configured by a placeholder JDBC-style URL, or `examples/local_postgres/run-wildcard.sh OUTPUT` for bounded table-qualified wildcard expansion. Run `examples/local_postgres/run-query.sh OUTPUT` for the reviewed query-file path.

Profile Through Trino

Database profiling is available in stable `1.6.0` and requires explicit read-only configuration, allowlists and resource budgets.

Install the optional Trino client and, when using MCP, the MCP SDK:

```
pip install "agent-paranoid-android[trino]"
pip install "agent-paranoid-android[mcp,trino]"
```

Stable `1.6.0` supports Basic, JWT, Kerberos, GSSAPI, OAuth2 and client certificates are documented in the configuration reference. Follow its runtime secret-indirection, HTTPS verification and local OAuth opt-in rules. Local contract tests do not establish remote authentication compatibility.

Configure an explicit read-only scope:

```
export TRINO_HOST=trino.example.internal
export TRINO_PORT=8443
export TRINO_USER=test_data_agent
export TRINO_HTTP_SCHEME=https
export TRINO_ALLOWED_CATALOGS=warehouse
export TRINO_ALLOWED_SCHEMAS=analytics
export TRINO_ALLOWED_TABLE_COLUMNS='warehouse.analytics.orders.*'
export TRINO_DEPLOYMENT_PROFILE=shared-hardened
export TRINO_MAX_INVOCATION_ESTIMATED_SCAN_BYTES=1073741824
```

`TRINO_ALLOWED_TABLE_COLUMNS` is optional for compatibility. When set, an entry is either one exact `catalog.schema.table.column` or the exact `catalog.schema.table.*` form. Wildcards require restricted mode and catalog and schema membership in the mandatory allowlists. Bounded `information_schema` metadata is validated and sorted before the first table aggregate; the complete column budget is charged in preflight. Executed profiling SQL still quotes explicit columns and never projects `*`.

The same endpoint and optional request defaults can come from a credential-free JDBC-style URL:

```
export TRINO_JDBC_URL='jdbc:trino://trino.example.internal:8443/warehouse/analytics?
SSL=true'
export TRINO_USER=test_data_agent
export TRINO_ALLOWED_CATALOGS=warehouse
export TRINO_ALLOWED_SCHEMAS=analytics
```

The catalog and schema path must match the allowlists. Credentials, tokens, roles, session properties, proxies, headers, unknown properties, and `SSL=false` fail before client construction. This is parsed into the Python Trino client; it does not

install or invoke Java or a JDBC driver. Explicit component settings may coexist only when their values agree.

The default aggregate-only MCP tools list allowlisted metadata, describe tables, and compute bounded aggregate profiles. They do not return source rows or raw category literals. Table and column profiling share cumulative statement, column, deadline, response, and optional scan budgets.

A wildcard authorizes aggregate profiling scope only. It does not authorize category literals, preserve-as-is, caller SQL stars, row-returning tools, provider payloads, default MCP literals, logs, or errors. Exact table-column entries retain their existing bounded category-summary policy.

Check that the Trino capability is installed, then start the MCP server:

```
test-data-agent doctor --require-extra trino
test-data-agent-mcp-trino
```

`doctor` checks the installed driver, safe-SQL support and configured deployment budget profile using a local smoke check. It does not validate the selected endpoint, authentication or catalog/schema allowlists, or contact Trino. Server startup and tool calls enforce their respective configuration and access checks; a successful local doctor check does not prove database access.

The explicit opt-in `run_safe_select` surface is disabled by default. Enabling it is a separate row-privacy contract: every string is recursively masked, but allowed non-string source values may remain, so its output is not source-free, anonymous, or a generated dataset.

Use the checked-in disposable example for a complete profile-to-generation workflow over Trino's synthetic TPC-H `tiny` catalog:

```
examples/local_trino/run.sh /tmp/agent-paranoid-trino-example
```

Use `examples/local_trino/run-jdbc.sh OUTPUT` to run the identical disposable workflow with its endpoint and `tpch/tiny` defaults supplied in JDBC syntax. Use `examples/local_trino/run-wildcard.sh OUTPUT` to exercise bounded `tpch.tiny.nation.*` expansion and deterministic field ordering. Use `examples/local_trino/run-query.sh OUTPUT` to profile one reviewed local `SELECT` as a virtual entity. Query-source mode additionally requires exact

`TRINO_ALLOWED_TABLE_COLUMNS` entries or a table-qualified wildcard and rejects unrestricted mode. It stores a fingerprint, not query text, literals, backend messages, endpoints, or result rows.

The typed Python entry point uses the same environment configuration and policy boundary:

```
from pathlib import Path

import trino

from test_data_agent import (
    SqlQueryAdapter,
    SqlQueryProfileRequest,
    profile_trino_query_source,
)
from test_data_agent.trino_config import TrinoConfig

request = SqlQueryProfileRequest(
    adapter=SqlQueryAdapter.TRINO,
    source_id="warehouse",
    entity="nation_query",
    query_file=Path("query.sql"),
)
profile = profile_trino_query_source(
    request,
    config=TrinoConfig.from_env(),
    driver=trino,
)
```

The example removes its container on success or failure and leaves only the safe profile, reviewed spec, generated rows, validation report, manifest, and a bounded result summary. See Configuration and MCP Tools for the full limits and tool contracts.

Transform Local Data

Use selective transformation when each output row must correspond to an input row. Unlike synthetic generation, the result has mixed origin and may contain explicitly replaced or approved preserved source values. Treat the destination as potentially sensitive; validation is not an anonymity certificate.

Choose An Action For Each Field

Action	Effect
`drop`	Remove the field.
`synthesize`	Generate new values using the declared generation policy.
`substitute`	Apply an explicit typed mapping, optionally shared by a domain.
`replace_text`	Apply explicit exact-text mappings. Field mappings override file-wide mappings without cascading.
`preserve`	Retain an eligible non-sensitive field only after exact local approval.

For an unmatched mapping, select rejection, synthesis or eligible preservation explicitly. A preserve fallback has the same approval requirements as direct preservation. Explicit mapped replacements may equal another source value, including a sensitive permutation; this is not permission for implicit copying. Mapping values never belong in logs, summaries, provider requests or MCP replies.

Review Before Execution

1. Prepare a valid behavior policy, source and any local mapping files. Use a new destination distinct from every input.
2. Run `transform-review SOURCE POLICY.yaml`. Review the value-free field decisions and record the displayed `snapshot_sha256`.
3. To edit an already valid policy locally, use `transform-review SOURCE POLICY.yaml --decide --edit-actions`, then `SAVE`. Review the saved bytes again. Saving is not approval.
4. If any action preserves values, obtain the receipt in your own interactive local terminal with `transform-approve SOURCE POLICY.yaml RECEIPT --snapshot-sha256 SHA`.
5. Execute with `transform-execute SOURCE POLICY.yaml DESTINATION --snapshot-sha256 SHA`, adding `--receipt RECEIPT` when required. Inspect provenance and validation.

`SOURCE`, `POLICY.yaml`, `RECEIPT`, `DESTINATION` and `SHA` above are placeholders, not a ready-to-run dataset approval. Use installed `COMMAND --help` for options. Never supply real data to a provider while preparing the policy.

Conditions For Preservation

Every preserved field needs an explicit non-sensitive decision, no positive or conflicting sensitivity evidence, and a bounded operator comment. Sensitive, unknown and disputed fields remain blocked. DECIMAL preservation is blocked by default. Approval binds the exact source, classification, policy and referenced mapping bytes. Any bound input change requires fresh review and approval.

Only the controlling local terminal can issue the receipt. Piped confirmation, an agent flag and MCP approval cannot replace it. A matching receipt permits only its reviewed plan; it does not authorize another dataset.

Multiple Inputs And Databases

Use a common saved profile with `transform-batch` for multiple inputs and final linked validation. Creation saves configuration only. Validation and execution consume the exact reviewed digest and any required local receipt.

Configured SQL sessions use administrator allowlists and read-only budgets to capture once. Later steps reuse the same frozen snapshot without reconnecting. Sessions enforce admission, expiry and cleanup. MCP consumers stay workspace confined and return metadata, not rows.

See the CLI contract for formats and arguments, MCP permissions for integration rights, and Configuration for schemas and budgets. Historical RC anchors remain valid links.

Integrations

Connect An MCP Client

These workflows describe stable `1.6.0`. Start with a workspace-confined server and enable only tools required for your task. For local preservation approval use Transform Local Data; MCP cannot issue it.

The project exposes two MCP servers with separate trust boundaries:

- the generator's ordinary file tools use one workspace; configured SQL transformation sessions additionally own private temporary captures;

- the Trino server's default aggregate-only tools provide allowlisted, read-only metadata and profiling.

Start with the generator server. Add Trino only when database profiling is required.

Before enabling tools, use MCP Permissions for the complete tool inventory, required filesystem/network/database rights, client confirmation policy, and separate transformation approval requirements. These are stdio servers: the client launches a local process; no inbound port or MCP OAuth scope is required. Client tool approval, operating-system access, database grants and human dataset review are separate controls.

For process and dependency isolation, use the separate hardened images in Container Deployment. The generator example has no network, while the Trino worker receives no generator workspace mount.

Generator-Only Quickstart

Install the optional MCP support in a dedicated environment. Use the version selected in Installation.

```
python3 -m venv /path/to/mcp-env
/path/to/mcp-env/bin/python -m pip install "agent-paranoid-android[mcp]"
mkdir -p /path/to/synthetic-workspace
```

Desktop clients may not inherit your shell's PATH. Configure the absolute executable path and restart the client after changing its configuration:

```
{
  "mcpServers": {
    "test-data-agent-generator": {
      "command": "/path/to/mcp-env/bin/test-data-agent-mcp-generator",
      "env": {
        "TEST_DATA_AGENT_WORKSPACE_ROOT": "/path/to/synthetic-workspace"
      }
    }
  }
}
```

Replace both absolute paths with your own. The client should discover `plan_dataset`, `inspect_dataset_plan`, `approve_dataset_plan`, and `recover_dataset_plan`. This local workflow requires no database or AI-provider credentials. The `mcpServers` example applies to clients supporting that configuration format; use your client's equivalent server settings otherwise.

Create `customers.csv` inside the workspace with this fictional fixture:

```
customer_id,email,status
1,alice@example.com,active
2,bob@example.com,paused
```

Ask your assistant: "Plan four synthetic customers from customers.csv with seed 81. Show me the specification for review before generating files." The corresponding `plan_dataset` arguments are:

```
{
  "source_path": "customers.csv",
  "workspace_path": "agent/customers",
  "source_type": "csv",
  "count": 4,
  "seed": 81,
  "output_format": "csv",
  "table_name": "customers"
}
```

The response has `approval_required: true` and `spec_path`. No generated dataset exists yet. Open the spec artifact locally and review field types, privacy, relationships, count, and seed. Inspection is a read-only operation:

```
{"workspace_path": "agent/customers"}
```

Call `inspect_dataset_plan` with those arguments. After a human reviews and explicitly approves the exact current specification, call `approve_dataset_plan` with:

```
{
  "workspace_path": "agent/customers",
  "reviewed_spec_sha256": "<review.current_spec_sha256 from inspection>"
}
```

Replace the placeholder with the actual fingerprint. If the specification changes, inspect and review it again; a copied fingerprint is not human approval. The approval response should report `row_counts: {"customers": 4}`, `validation_valid: true`, and paths to output, manifest, validation report, and approval receipt. Inspect the same workspace again to confirm `phase: completed`. Report these summaries and paths; do not attach or inline dataset rows.

Resume An Existing Synthetic Plan

If the client closes after planning, keep the agent workspace and resume the same conversation through the client's normal session-resume action. Restore the same verified installation, workspace root, MCP connection and tool filter. Keep the existing model, authentication, sandbox and confirmation policy. Run only one copy of that client session.

For the Codex CLI, the upstream client command is `codex resume SESSION_ID`; replace `SESSION_ID` with the existing conversation's identifier. This is a Codex command, not an Agent Paranoid Android option. If the original launch supplied MCP settings only for that invocation, reapply those same settings through the reviewed client launcher when resuming. A bare resume command does not establish that the required server and tool filter were restored.

Continue with `inspect_dataset_plan` for the existing workspace. Do not replay `plan_dataset`, delete the plan, or choose a new workspace merely to clear a non-empty-directory refusal. A launcher refusing to create a second plan does not establish that the package or final-commit checks failed.

When the phase is `awaiting_approval`:

1. Inspect the current plan and make the entire current `dataset_spec.yaml` available for the human to open and read. Check both its file bytes and `review.current_spec_sha256` against the specification being reviewed.
2. Obtain fresh, explicit human consent for that exact current specification. A model's summary, copied fingerprint, bootstrap prompt, general release authorization or consent sent before the final specification was prepared does not complete this review.
3. Immediately before approval, inspect and check the same file again. Changed bytes require another full-file view and renewed consent.
4. Call `approve_dataset_plan` with the unchanged reviewed fingerprint, then `validate_dataset` and inspect the completed workspace. Report validation, counts and artifact paths without returning rows.

Client-session resume and `recover_dataset_plan` are different operations. For an interrupted approval, follow the state returned by inspection: report an existing `completed` result, use recovery only when `next_action` requests it, and stop for inspection on an unexpected state. Resuming a conversation does not itself approve a dataset or authorize preservation. See the permission boundaries.

Corrective Next Steps

- Missing executable or MCP extra: check the absolute path, install `[mcp]` into that environment, and restart the client.
- Rejected workspace path: move the input inside the configured root and use relative paths. Do not expand permissions to bypass the boundary.
- Non-empty planning directory: inspect the existing plan and resume its review. Use a new workspace only when deliberately starting a separate dataset.

- Stale review fingerprint: inspect, review the changed specification, and obtain renewed human approval before submitting its current fingerprint.
- Interrupted approval or a lost response: inspect first. If `phase` is `completed`, report the existing result. If `next_action` is `recover`, call `recover_dataset_plan` with the workspace and reviewed fingerprint. Recovery retains published outputs without generating another dataset. If inspection requests review and approval, follow that state instead.
- Fixed argument-validation or transport-limit error: correct arguments against the discovered tool schema or reduce request size. Do not disable budgets or expect rejected values to be reflected in error messages.

Add Trino only for database profiling, using the separate configuration and Safe Trino Sequence below. Keep database credentials in the Trino server for that aggregate-profile-to-synthetic workflow. The separately configured SQL transformation session has its own connection requirements.

Prepare A Workspace

Common-Profile Workflow

Stable `1.6.0` provides `common_transformation` in the generator console server. Registration grants no dataset permission or preservation approval. Stable acceptance has finite recorded limits; changed activation still needs its own review. It accepts `operation` (`review`, `validate`, `execute`), workspace-relative `profile`, and positive explicit `max_total_bytes`, `max_review_bytes`, `max_output_bytes`. Validation/execution require the exact reviewed `snapshot_sha256`. Preservation additionally requires a pre-existing local `receipt`; MCP cannot issue it. Optional `destination` is execute-only and names a new direct-child output bundle. Without it, execution is temporary.

The workspace root comes from server configuration, never tool arguments. Responses contain value-free review/summary metadata, not output rows or mapping literals. Existing transport/work budgets still apply; these byte settings do not enlarge source-free profiling or serialized response budgets. Unknown arguments, including `root` and `approved`, are rejected without reflection. Configuration creation and human confirmation remain local CLI operations. Installed fictional acceptance with MCP SDK1/SDK2 does not authorize private data access or issue a local preservation receipt.

```
mkdir -p /path/to/synthetic-workspace
```

Inputs, safe profiles, reviewed specs, rules, and outputs used through generator file tools must remain below this directory. Server-owned temporary storage, audit logs and authentication files have separate administrator-configured locations. The workspace setting is an application path boundary, not an operating-system sandbox.

MCP Client Configuration

Use the installed console commands:

```
{
  "mcpServers": {
    "test-data-agent-generator": {
      "command": "test-data-agent-mcp-generator",
      "env": {
        "TEST_DATA_AGENT_WORKSPACE_ROOT": "/path/to/synthetic-workspace",
        "TEST_DATA_AGENT_MCP_MAX_INVOCATION_SECONDS": "120"
      }
    },
    "test-data-agent-trino": {
      "command": "test-data-agent-mcp-trino",
      "env": {
        "TRINO_HOST": "trino.example.internal",
        "TRINO_PORT": "443",
        "TRINO_USER": "synthetic_data_reader",
        "TRINO_HTTP_SCHEME": "https",
        "TRINO_ALLOWED_CATALOGS": "hive,iceberg",
        "TRINO_ALLOWED_SCHEMAS": "test_data,staging",
        "TRINO_QUERY_MAX_EXECUTION_TIME": "30s",
        "TRINO_QUERY_MAX_RUN_TIME": "45s",
        "TRINO_QUERY_MAX_SCAN_PHYSICAL_BYTES": "1GB"
      }
    }
  }
}
```

Give the Trino identity only the required metadata and SELECT access through database grants; no write or administration rights are needed. Catalog/schema allowlists do not create those grants. `TRINO_ALLOWED_TABLE_COLUMNS` narrows safe table profiling, but is not a universal per-tool ACL for metadata, counts, rule probes or optional `run_safe_select`. Use database grants for strict table/column isolation; see the permission matrix.

Do not place a password or token directly in a committed MCP configuration. Use the client's secret mechanism or an environment injected by the runtime.

Safe Generator Sequence

For selective transformation review, call `review_transformation` with workspace-relative `input_path`, `policy_path` and optional `table_name`. It reads a fixed local snapshot in the format selected by the saved policy and returns `review_only`, `snapshot_sha256` and the same value-free review as CLI `transform-review`.

Referenced mapping files stay local to the policy directory. The tool writes no artifacts, creates no approval receipt, executes no transformation and connects to no database. It is separate from ordinary source-free generator operations.

The stable `1.6.0` generator registers `execute_transformation` with workspace-relative `input_path`, `policy_path`, new `output_path`, mandatory `snapshot_sha256`, optional `table_name`/existing `receipt_path`, and optional `max_total_input_bytes`/`max_output_bytes` run caps. It consumes the same exact snapshots and budgets as CLI execution and returns `transformation_completed`, digest, provenance and budget summaries, never dataset rows or mapping literals. Explicit replacements may permute sensitive source values under the same field-independent contract. Review retains a value-free sensitivity note; mapped values stay in the selected local artifact, never MCP responses. This does not grant direct preservation or an implicit copy fallback.

Preservation requires a matching receipt issued separately by the human local controlling-terminal CLI. There is no MCP receipt issuer or approval flag. Changed inputs invalidate receipts; workspace escapes, existing destinations, unsupported actions and budget violations fail closed without copying fallback. Registration does not authorize a dataset. Changed activation requires fresh independent safety review and executable evidence. Changed public activation still requires matching contracts, executable safety evidence and independent exact-SHA review. A receipt never authorizes database access or external APIs.

1. Put a CSV file, CSV folder, or safe profile below the workspace root.
2. Call `plan_dataset` with that source, a new agent workspace, count, seed, and output format.
3. Stop and review the written `dataset_spec.yaml`.
4. Call `inspect_dataset_plan` and record `review.current_spec_sha256`.
5. Call `approve_dataset_plan` with that exact fingerprint only after review.
6. Report summaries and artifact paths, not generated rows.

Use `profile_csv`, `infer_dataset_spec`, `generate_dataset`, and `validate_dataset` separately when an advanced client needs control over each pipeline stage. Direct `generate_dataset` and `export_dataset` do not require a planning approval argument; configure the client's confirmation policy before enabling these writing tools. Permission to invoke `approve_dataset_plan` is not evidence that the human reviewed the displayed specification.

For the planning/approval workflow, embed `business_rules` in the reviewed DatasetSpec. Direct `generate_dataset` and `export_dataset` calls can instead provide `business_rules_path` or a bounded structured `business_rules_payload`. Use one rule source: external rules cannot be combined with embedded rules.

Generator MCP reads newline-framed requests through the same bounded transport used by Trino MCP. Each request receives a fresh non-resettable budget. The default boundary rejects raw frames above 1 MiB, JSON deeper than 100 levels, JSON with more than 10,000 structural nodes, or an individual scalar above 256 KiB before MCP/Pydantic materialization. Complete JSON-RPC responses are limited to 4 MiB and overflow becomes a fixed local error. Typed argument-validation failures also become the fixed `Tool arguments failed validation` error; rejected caller values and nested Pydantic diagnostics are not returned. Malformed typed requests are rejected before SDK dispatch with fixed `Invalid request parameters` text, and malformed notifications are dropped; their caller-controlled values do not enter SDK logs or retained exceptions. Each server process admits at most 32 active requests. Excess requests receive a fixed bounded capacity error, and disconnect or teardown clears retained request state. Trino execution is separately capped at eight concurrent operations per process.

Safe Trino Sequence

The default aggregate-only tools are source-literal-free and contain only metadata and aggregate profiling operations:

- `list_catalogs`, `list_schemas`, `list_tables`, `describe_table`;
- `profile_table`, `profile_table_safe`, `profile_column`;
- `profile_foreign_key`, `profile_temporal_ordering`, `profile_formula_rule`;
- `profile_conditional_required`, `profile_conditional_allowed_values`;
- `profile_aggregate_mapping`.

Catalog and schema discovery returns only names present in the configured allowlists. Tables may be listed only after their catalog and schema pass those allowlists. Backend-controlled driver and enumeration failures are exposed as the fixed `Trino request failed` error without retaining the original text.

This default aggregate-only surface has no row-returning diagnostic. Its successful responses, validation and database errors, and metadata-only audit records do not contain source-cell literals.

1. Call `list_catalogs`, `list_schemas`, and `list_tables`.
2. Call `describe_table`.
3. Call `profile_table_safe` for an allowlisted table.
4. Pass that response to generator `plan_trino_dataset` with a new workspace, explicit count, seed, and output format.
5. Stop and review the written `dataset_spec.yaml`.
6. Call `inspect_dataset_plan` and record `review.current_spec_sha256`.
7. Call `approve_dataset_plan` with that value as `reviewed_spec_sha256` only after the human reviewed that exact spec fingerprint.
8. Do not export or relay source rows.

Both catalog and schema allowlists are mandatory by default. HTTPS is the default. Plain HTTP requires an explicit override and is intended only for an isolated local Trino instance.

The explicit opt-in row-returning tool `run_safe_select` is not exposed by default. Trusted clients that need it must set `TRINO_ENABLE_SAFE_SELECT=true`. The review-first planning sequence above does not require it. Its bounded row-shaped result masks every string, including names and addresses missed by heuristic classification, plus non-string fields or values recognized as sensitive. Other non-string source values may remain, so enabling it does not make returned rows source-free, anonymous, or suitable for relay to an LLM or generated output.

Expected Result

The default generator and default aggregate-only Trino tools return compact metadata:

```
rows: customers=25, orders=25
seed: 12345
validation: passed
synthetic: true
source rows copied: false
```

Generated files stay in the workspace. These default tools do not return dataset or source rows; explicit opt-in row-returning tools are outside this expected result.

Failure Conditions

The server rejects:

- paths outside the workspace, including existing symlink escapes;

- existing output files and non-empty output directories;
- unrestricted SQL, DDL, DML, joins, CTEs, and subqueries;
- likely PII projections and raw sensitive rule literals;
- non-Trino or oversized inline planning profiles;
- DatasetSpec input passed to `plan_dataset` instead of `generate_dataset`;
- missing Trino allowlists;
- requests exceeding configured input, output, query, or execution limits.

See MCP Tools and Configuration for details.

Preservation approval is never an MCP capability. The registered separate workspace execution consumer cannot issue local receipts, return rows or broaden source-free generator authority. Changed public activation requires the scoped safety review and executable evidence specified by ADR-0020 and ADR-0021.

Configured SQL Sessions

The running generator server registers `configured_query_session` with an instance-owned session manager. Use only an authorized administrator-configured PostgreSQL or Trino source. A saved batch profile and bounded query references are workspace files; tool arguments never select endpoints or credentials.

Operation	Arguments	Result and lifetime
`open`	`profile`, `references`, positive `max_total_bytes`, `max_review_bytes`, `max_output_bytes`	Capture once; return metadata review, opaque `handle`, digest, expiry and local approval descriptor.
`review`	`handle`	Reuse the same frozen batch; no database reconnection.
`validate`	`handle`, exact `snapshot_sha256`	Validate temporary output; retain the session until expiry or close.
`execute`	`handle`, exact `snapshot_sha256`, new direct-child `destination`	Verify any required local receipt, validate and publish; close the session.
`close`	`handle`	Remove owned snapshots and receipts; the handle cannot be reused.

Responses contain review/summary metadata, never captured rows, query text, mapping literals or connection secrets. `local_approval` contains an owned temporary root, `batch.yaml`, a fixed receipt filename, digest and input/review limits. These references help the local operator invoke the existing CLI:

```
test-data-agent transform-batch approve /owned/root batch.yaml local-session-
receipt.json \
```

```
  --snapshot-sha256 REVIEWED_DIGEST \
  --max-total-input-bytes REVIEWED_INPUT_LIMIT --max-review-bytes REVIEWED_REVIEW_LIMIT
```

Use the exact metadata values while the session is alive. The CLI opens the controlling terminal for confirmation; MCP has no approval operation or boolean. This descriptor itself grants no authority. Direct or fallback preservation requires a valid exact receipt; explicit mappings retain their separate safety checks. Changed local bytes cannot produce a receipt for the frozen batch.

Expiry, refusal and shutdown remove owned inputs and receipts. Execution retains only the selected validated output bundle inside the parent workspace. Expired or closed handles refuse without silently recapturing. Start a new explicit session and review its new digest if another capture is needed. Cumulative byte admission is not refunded on close or failure; restart the owning server only after resolving the refusal and reviewing its configured limits.

Cleanup failure returns a fixed incomplete-cleanup error and latches the owner unhealthy; further work refuses. Preserve the failure evidence and inspect local temporary storage before restarting. Default bounds and overrides are documented in configuration. The entrypoint closes all sessions in a shutdown handler even if transport fails. Custom SDK embedding must explicitly inject a session owner and close it; the module-level SDK object creates no SQL sessions.

AI Integration

Model-specific Python integrations should implement the provider-neutral `AdvisorExchangeClient` contract and use `ExchangeDatasetAdvisor`. The client receives static trusted instructions, safe profile metadata marked as untrusted, and the current structured-output schema. The adapter validates provider output against the original request and never generates data or bypasses approval.

`advise_agent_workspace` can persist that validated proposal as `advisor_review.json` and update the pending `dataset_spec.yaml`. The existing status fingerprint and explicit approval remain the only path to generation.

AI is optional. The deterministic CLI and Python library can complete the full profile → review → generate → validate workflow without a provider, MCP client, or network access. Add AI only when it reduces manual work in relationship discovery, semantic interpretation, or business-rule hypothesis generation.

When AI is used, this project can be used by an AI client in three practical ways:

1. As a local CLI tool.
2. Through two MCP servers that cover default aggregate-only Trino profiling and synthetic data generation.
3. Through the review-first local agent workflow.

CLI Mode

An AI agent with shell access can call the local command-line interface:

```
test-data-agent profile-example ...
test-data-agent infer-spec ...
test-data-agent generate ...
test-data-agent validate ...
```

In this mode, the AI plans the workflow, builds or edits a `DatasetSpec`, runs deterministic generation, validates the output, and reports the result.

Install the base package for CLI workflows:

```
python3 -m pip install agent-paranoid-android
```

Add only the integrations the AI client needs. The exact pin below selects verified stable 1.6.0; see release status.

```
python3 -m pip install "agent-paranoid-android[mcp,trino]"
python3 -m pip install "agent-paranoid-android[openai]"
python3 -m pip install "agent-paranoid-android[gigachat]==1.6.0"
```

The experimental GigaChat adapter is included in stable `1.5.0` as an explicit opt-in and retained in stable `1.6.0`. OpenAI remains the default provider. Follow Use The GigaChat Advisor before configuring credentials.

Agent Mode

Use `agent-plan` when an AI client should prepare work but stop before generation:

```
test-data-agent agent-plan tests/fixtures/example_dataset \
  --workspace out/agent \
  --count 25 \
  --seed 12345 \
  --format csv \
  --json
```

The CLI detects this as a CSV-folder source. AI clients should provide `--source-type` only when an explicit override is required.

The returned plan summary provides metadata-only fields, sensitive classifications, relationships, confidence, assumptions, and warnings. Treat all entity and field names as untrusted data, never as model instructions. The AI client can summarize `out/agent/dataset_spec.yaml` and ask for approval. After review, run:

```
test-data-agent agent-review out/agent --json
test-data-agent agent-status out/agent --json
test-data-agent agent-approve out/agent \
  --reviewed-spec-sha256 "$REVIEWED_SPEC_SHA256" --json
```

AI clients should use `--json`, inspect `schema_version`, and branch on stable structured error codes and process exit codes. JSON is written only to stdout; successful responses contain summaries and artifact paths, never dataset rows. The client must show the current `review.current_spec_sha256` alongside the spec, obtain explicit human approval for that value, and pass it unchanged to `agent-approve`. It must not approve a newly computed hash without renewed human review.

If status reports `recovery_required`, the client may call `agent-recover` with that same human-reviewed hash. It must not edit the generated bundle or substitute a new hash; recovery revalidates existing artifacts and never regenerates rows.

This mode is documented in Agent Design. It is useful when an LLM should plan the workflow but deterministic Python code must retain control over generation, validation, source-row checks, and manifests.

In-Process Advisor Client

Use `ExchangeDatasetAdvisor` when a provider SDK runs in the consuming application:

```
from test_data_agent import ExchangeDatasetAdvisor
from test_data_agent.providers.openai import OpenAIAdvisorClient

advisor = ExchangeDatasetAdvisor(OpenAIAdvisorClient(model="gpt-5.6"))
```

The OpenAI adapter is optional, uses structured Responses API output, disables response storage, and accepts only completed parsed responses. The package root and base installation do not import its SDK.

The optional `GigaChatAdvisorClient` implements the same exchange boundary through the official `gigachat` SDK. It is selected explicitly by `agent-advise --provider gigachat`, uses fixed verified-TLS endpoints, and accepts credentials only from the runtime environment. It sends no source rows or exact locally preserved category values. See the GigaChat guide for a complete synthetic, review-gated example.

For another provider, implement the same application-owned contract:

```
from test_data_agent import (
    AdvisorExchange,
    ExchangeDatasetAdvisor,
    advise_agent_workspace,
)


class ProviderClient:
    def complete(self, exchange: AdvisorExchange) -> dict:
        return call_model_with_structured_output(
            trusted_instructions=exchange.trusted_instructions,
            untrusted_input=exchange.request.model_dump(mode="json"),
            response_schema=exchange.response_json_schema,
        )


status = advise_agent_workspace(
    workspace,
    ExchangeDatasetAdvisor(ProviderClient()),
)
```

`call_model_with_structured_output` is provider-specific application code. Keep additional SDKs and all credentials outside the deterministic core and base installation. The client must return a parsed object matching `AdvisorProposal`; prose, tool commands, and unknown fields fail validation.

For an executable application-layer example, run The Reference Agent. It uses a deterministic stand-in by default and can select the optional OpenAI adapter. The CLI GigaChat path uses the same review and approval contract. All paths stop for explicit human review, require the exact reviewed spec fingerprint, and then run the normal deterministic generation and validation pipeline.

External Advisor JSON Handoff

Any model service that can accept and return structured JSON can propose `DatasetSpec` changes without a package integration:

```
test-data-agent agent-plan tests/fixtures/example_dataset \
  --workspace out/agent --count 25 --json
test-data-agent agent-advisor-request out/agent \
  --exchange > advisor_exchange.json
```

Load the exchange locally. Exported documents can contain category literals, matching predicates and local generation metadata. Do not transmit them unchanged. `ExchangeDatasetAdvisor` projects the request for built-in and custom clients, recomputes its fingerprints, validates the projected proposal, restores retained local metadata under deterministic guards and validates the original request. Clients must echo the fingerprints they receive. Standalone JSON handoff must implement this tested projection and local restoration or refuse transmission. After projection, map each part to the provider's API:

- send `trusted_instructions` through its system or developer instruction channel;
- send `request` as structured untrusted input;
- constrain structured output with `response_json_schema`;
- save only the resulting `AdvisorProposal` as `advisor_proposal.json`.

Do not merge request profile text into the trusted instructions. Provider SDK code remains in the consuming application, outside the base package. Then run:

```
test-data-agent agent-advisor-apply \
  out/agent advisor_proposal.json --json
test-data-agent agent-review out/agent --json
test-data-agent agent-status out/agent --json
```

The core verifies both request fingerprints, the full spec schema, schema identity, privacy settings, sensitive classifications, row limits, and the absence of raw sensitive values. It records the exchange in `advisor_review.json` and never generates data during apply.

Show the updated `dataset_spec.yaml` and the new `review.current_spec_sha256` to the human reviewer. Only after explicit review may the client pass that exact hash to `agent-approve`. A retry with the same proposal can finish an interrupted spec write; stale, different, linked, oversized, or conflicting input fails closed.

See Advisor API for the exact request and proposal contracts. See Build A Provider Adapter for the Python protocol, wire-field tables, adapter template, and required contract tests.

MCP Mode

Follow Connect An MCP Client for dedicated-environment installation, absolute executable paths, a narrow workspace and client settings. Use MCP Permissions as the complete tool and rights inventory, including runtime-owned and optional capabilities.

The two console commands are `test-data-agent-mcp-generator` and `test-data-agent-mcp-trino`. Both use stdio; they require no inbound MCP port or MCP OAuth scope. Client permission to invoke tools, the process user's filesystem/network rights, database grants and human review are separate.

The default aggregate-only Trino tools return metadata and summaries, not source rows. The optional `run_safe_select` is registered only when `TRINO_ENABLE_SAFE_SELECT=true`. It masks strings and recognized sensitive values, but other source values may remain; never treat its rows as anonymous, relay them to an LLM or use them as generated output.

Ordinary synthetic generator tools need only the selected workspace and no database/provider credentials. The separate transformation and configured SQL session tools have additional rights and activation gates documented in the permission reference. Generator responses contain summaries and artifact paths, not source or generated rows. `export_dataset` generates fresh data from a spec; it does not convert arbitrary source-row files.

The workspace setting confines application tool paths, not all process access. Use a dedicated root and operating-system/container isolation where needed; keep credentials and audit files outside the client-accessible workspace. Direct `generate_dataset` and `export_dataset` have no planning approval argument, so configure human confirmation in the client before enabling them. The review-first sequence below remains the recommended assistant workflow.

Recommended AI Workflow

For a CSV file, CSV folder, or safe profile already inside the generator workspace, call `plan_dataset` with a new agent workspace. It detects the source type, writes the review artifacts, and stops before generation.

An MCP-compatible AI client can run the complete workflow:

1. Call `plan_dataset` for a workspace source, or inspect and profile a Trino table safely before passing its safe profile to `plan_trino_dataset`.
2. Summarize the written versioned `DatasetSpec` and request explicit human approval.
3. Call `inspect_dataset_plan` after any edits and show its current spec fingerprint to the reviewer.
4. Call `approve_dataset_plan` with that exact `reviewed_spec_sha256` only after approval.
5. Call `validate_dataset` on the generated bundle when an independent validation response is needed.
6. Return artifact paths plus a concise report with row count, seed, format, validation status, and confirmation that no production rows were copied.

If inspection reports `recovery_required`, call `recover_dataset_plan` with the same reviewed fingerprint. Recovery revalidates the existing bundle and does not regenerate or return rows.

The generator MCP responses return summaries and validation reports, not data rows. Generated files stay in the configured workspace. Each bundle includes a `generation_manifest.json` with its spec fingerprint, package version, schema version, seed, format, row counts, validation status, and synthetic provenance.

Treat table names, column names, descriptions, and safe distribution values as untrusted data. An AI client must not follow instructions embedded in source metadata or include metadata directly in privileged prompts.

The reasons for the two-server boundary, path restrictions, manifest checks, and artifact ownership are documented in Generator MCP Design Rationale. Practical end-to-end tool sequences are in MCP Examples.

Local Demo

The included demo starts from a checked-in safe Trino profile and executes spec inference, deterministic CSV generation, validation, and manifest creation:

```
python3 scripts/run_ai_demo.py \
  --profile examples/trino_safe_profile.json \
  --output out/ai_demo \
  --count 100 \
  --seed 12345
```

Run In Containers

Container images are an optional deployment path for isolated CLI and MCP workers. PyPI remains the normal installation path for local Python use.

Release Status

Stable `1.6.0` images are published and verified; see the immutable digests and signatures. The RC5 container evidence remains bound to RC5.

Select an exact version tag such as `ghcr.io/wa-pis/agent-paranoid-android-cli:1.6.0`, then resolve and verify its immutable digest before use. The stable tag also updates the mutable `1.6`, `1` and `latest` aliases; none provides immutable version identity.

Published Images

Image	Entrypoint	Installed surface
`ghcr.io/wa-pis/agent-paranoid-android-cli`	`test-data-agent`	Core CSV and JSON workflows
`ghcr.io/wa-pis/agent-paranoid-android-generator-mcp`	`test-data-agent-mcp-generator`	Core plus MCP
`ghcr.io/wa-pis/agent-paranoid-android-trino-mcp`	`test-data-agent-mcp-trino`	Core, MCP, Trino, and SQLGlot

Release tags publish `linux/amd64` and `linux/arm64` manifests. PyArrow is not installed in these images. Use the PyPI `parquet` extra when Parquet output is required.

Pull requests build and run every target on both architectures. ARM64 checks use QEMU only as CI execution support; the resulting image still reports `arm64` and must pass the same non-root, read-only, network-isolated health contract as the native AMD64 validation. Both architectures also run actual installed CLI generation, validation and seeded replay in JSON, CSV and SQL; generator MCP profiling, inference, generation, validation and export over stdio; and Trino MCP discovery plus an excluded-catalog refusal before network access. Evidence records the source commit and image digest. The minimal images also prove a safe missing-Parquet-extra refusal. Live database acquisition is a separate disposable-database campaign, not a health-check claim.

Build Locally

Build only the target you need:

```
docker build --target cli -t agent-paranoid-android:cli .
docker build --target generator-mcp -t agent-paranoid-android:generator-mcp .
docker build --target trino-mcp -t agent-paranoid-android:trino-mcp .
```

The Dockerfile uses digest-pinned Python and uv images, a frozen lockfile, and separate dependency sets. Final images run as UID/GID `65532` by default. The shared runtime layer installs fixed Bookworm PCRE2 and Perl packages and checks their versions during the build. Perl remains a Debian essential package; image scans continue to block fixable High and Critical findings.

Prepare Compose

Create writable workspace and audit folders plus a private audit key file:

```
umask 077
mkdir -p workspace audit/generator audit/trino secrets
openssl rand -base64 32 > secrets/generator_audit_hmac_key
openssl rand -base64 32 > secrets/trino_audit_hmac_key
chmod 600 secrets/generator_audit_hmac_key secrets/trino_audit_hmac_key
export CONTAINER_UID="$(id -u)"
export CONTAINER_GID="$(id -g)"
```

`CONTAINER_UID` and `CONTAINER_GID` must remain non-zero. They let the non-root process write to bind-mounted host folders on Linux. Never commit the `secrets`, `audit`, or `workspace` folders.

Build the Compose targets from the current checkout:

```
docker compose build cli generator-mcp trino-mcp
docker compose run --rm cli doctor --skip-smoke
```

The Compose contract applies a read-only root filesystem, drops all capabilities, enables `no-new-privileges`, limits CPU, memory, processes, and temporary storage, and publishes no ports.

Generator MCP

The generator receives only `/workspace` and `/audit`; it has no network:

```
docker compose run --rm -T generator-mcp
```

Set `TEST_DATA_AGENT_MCP_MAX_INVOCATION_SECONDS` in the shell or Compose `.env` file before starting generator MCP to configure its invocation deadline. The default is 120 seconds; finite positive values are required. Independent generation, profiling, transformation and container memory limits still apply.

An MCP client can launch that command directly. Use an absolute Compose path:

```
{
  "mcpServers": {
    "test-data-generator": {
      "command": "docker",
      "args": [
        "compose",
        "-f",
        "/absolute/path/agent-paranoid-android/compose.yaml",
        "run",
        "--rm",
        "-T",
        "generator-mcp"
      ]
    }
  }
}
```

The client receives MCP messages over stdio. There is intentionally no exposed HTTP port.

Trino MCP

Set an HTTPS endpoint and narrow allowlists before starting the Trino worker:

```
export TRINO_HOST=trino.example.internal
export TRINO_PORT=443
export TRINO_USER=test_data_agent
export TRINO_ALLOWED_CATALOGS=iceberg
export TRINO_ALLOWED_SCHEMAS=dev,staging
docker compose run --rm -T trino-mcp
```

Missing host or allowlists fail before the server starts. The Trino service has network access but receives no generator workspace mount. The Compose example does not enable unrestricted access, plain HTTP, or the optional raw-SQL tool.

Audit Secret

Compose mounts separate generator and Trino audit keys below `/run/secrets`. Keys are not stored in image metadata or process environment. The application rejects a secret file that is empty, oversized, writable by another user, linked, or configured together with `TEST_DATA_AGENT_AUDIT_HMAC_KEY`.

The services also use separate host directories and logs: `audit/generator/generator.jsonl` and `audit/trino/trino.jsonl`. A compromise of one worker therefore does not grant the key or writable log mount of the other. Stop the corresponding writer before rotating either file. See MCP Audit Logging.

Verify A Published Image

Use the immutable digest shown by GHCR rather than relying on a mutable tag:

```
IMAGE=ghcr.io/wa-pis/agent-paranoid-android-generator-mcp
DIGEST=sha256:replace-with-published-digest

cosign verify \
  --certificate-identity-regexp \
  '^https://github.com/wa-pis/agent-paranoid-
android/.github/workflows/containers.yml@refs/tags/v[0-9].*$' \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com \
  "${IMAGE}@${DIGEST}"

gh attestation verify "oci://${IMAGE}@${DIGEST}" \
  --repo wa-pis/agent-paranoid-android

docker buildx imagetools inspect "${IMAGE}@${DIGEST}" \
  --format '{{ json .SBOM }}'
```

The release workflow signs the multi-platform manifest digest with a short-lived GitHub OIDC identity and publishes BuildKit SBOM and provenance attestations. No Cosign private key is stored in the repository.

Pull-request CI also scans every native container target for fixable High and Critical operating-system and Python-package vulnerabilities. A finding at either severity blocks publication through the container validation gate. Unfixed upstream findings remain visible in the scan output and must be reviewed separately during release-candidate security review.

Health Semantics

The image health command performs only local checks: non-root execution, expected dependencies, workspace access, Trino safety configuration, and audit configuration. It never connects to Trino or reads a dataset. Because MCP uses stdio, container liveness is also represented by the MCP process itself.

Reference

Reference Index

Start with a task guide when you need a complete workflow. Use these references for exact options, schemas and integration contracts.

Commands And Automation

- CLI command index and workflow recipes
- JSON automation, errors and exit codes, shell completion
- Configuration and budgets, resource limits

Data Contract

- Source/output matrix and generation modes
- DatasetSpec fields, schema compatibility
- CSV profiling details
- Relationships
- Migration notes and compatibility aliases
- Public stability, runtime support, dependency compatibility

Integrations

- MCP tools, permissions, audit logging
- Advisor API, GigaChat, custom advisor, semantic provider, reference agent

Further Reading

- Threat model
- Product validation pilot
- Dataset workflow
- History and maintenance

Choose An Approach

Agent Paranoid Android is for teams that need deterministic test datasets from bounded CSV, PostgreSQL, or Trino evidence without copying source rows. Its primary interfaces are the CLI and Python library. Database clients, MCP, and AI providers are optional integrations around the same review and validation boundaries.

The database workflows below are available in stable `1.6.0`.

Quick Comparison

Need	Best starting point	Why
A safe, repeatable dataset from profiled structure and reviewed rules	Agent Paranoid Android	Profiles bounded evidence, generates fresh values, validates executable rules, and records a seed and manifest.
One PostgreSQL table or Trino table with a known schema	Exact component configuration and exact table/column allowlists	This is the stable, narrowest database-source path. It keeps executed profiling work explicit and aggregate-only.
A platform-supplied JDBC connection string	JDBC-style endpoint input plus separate credentials, allowlists, and budgets	It reuses the Python adapter and accepts endpoint syntax only; it does not add Java or a JDBC driver.
Every column in one approved table	Qualified column wildcard	`schema.table.*` or `catalog.schema.table.*` expands through bounded metadata into explicit columns; executed SQL never projects `*`.
One reviewed derived relation from a single table	`profile-query`	A strict local-file `SELECT` becomes one aggregate-only virtual profile. It is not an arbitrary SQL runner and never returns query rows.
A few hand-authored values or one simple object factory	Faker or an application fixture factory	Less setup when you already know every field and do not need profiling, review artifacts, or dataset validation.
Application-native fixtures coupled to ORM models and lifecycle hooks	A framework fixture factory	Better fit when model constructors and database callbacks are the contract being tested.
Production-like statistical fidelity with formal privacy targets	A specialist statistical synthesizer plus a privacy evaluation	This project does not certify anonymity, differential privacy, or resistance to every re-identification attack.
A masked or reduced copy of production	A governed masking/subsetting system	Use only when policy permits retaining real rows; this project deliberately generates fresh rows instead.

Choose This Project When

- source access must stay read-only and metadata-focused;
- generated rows must be reproducible from an explicit seed;
- foreign keys, temporal order, formulas, and business rules must be executable and independently validated;
- humans must approve inferred specifications before generation;

- CSV, JSON, SQL, or Parquet output needs an evidence manifest.

Start with the offline installed demo:

```
test-data-agent demo --output out/demo
```

Choose Something Else When

Use a simpler factory when manually defining every field is cheaper than profiling and review. Use a specialist privacy system when you need a measured privacy guarantee. Use governed masking only when retaining source rows is an explicit requirement and is allowed by your data policy.

No approach removes the need to review sensitive identifiers, rare free text, quasi-identifiers, and organization-specific privacy requirements.

Profiles And Specs

Profiles and specifications have different roles.

Safe Profile

A profile describes source structure without becoming a copy of the source dataset. It can contain:

- field names and inferred types;
- null ratios and approximate distinct counts;
- numeric ranges and percentiles;
- date and timestamp ranges;
- safe distributions for non-sensitive low-cardinality fields;
- string length statistics and masked sensitive patterns;
- inferred relationship and constraint candidates.

A profile must not contain source rows, raw PII, credentials, tokens, or rare free-text values. Exact values are disabled by default. An explicit field-scoped local allowlist may retain a reviewed bounded non-sensitive business enum or constant after content, cardinality, and value-length checks.

Create one from a CSV folder:

```
test-data-agent profile-example data/example_dataset \
  --output out/profile.json
```

For example, retain a reviewed local enum while replacing every other source literal:

```
test-data-agent profile-example data/example_dataset \
  --local-category orders.status \
  --output out/profile.json
```

This authorization remains local. External advisors receive deterministic field-scoped labels, and default MCP responses do not receive the exact enum.

SQL Query Source

This source type is available in stable `1.6.0`.

One reviewed local query file can shape a PostgreSQL or Trino virtual entity:

```
test-data-agent profile-query query.sql \
  --adapter postgres \
  --source-id warehouse \
  --entity paid_orders \
  --output out/profile.json
```

SQL source policy 1.1 accepts exactly one fully qualified, single-table `SELECT` with explicit projections or an authorized qualified wildcard, bounded filters, and a small deterministic scalar-expression set. It rejects joins, CTEs, subqueries, set operations, windows, table functions, commands, volatile or unknown functions, multiple statements, and unauthorized references before derived aggregate work begins.

It also accepts aliased `SUM`, `COUNT`, `MIN`, `MAX` and `AVG` projections, optionally grouped by explicit authorized source columns. Each other selected column must be a grouping key. `COUNT(*)` counts rows; it is not a row wildcard. Aggregate arguments may use the existing scalar expressions, but nested or wrapped aggregates, DISTINCT aggregates, HAVING, grouping expressions/ordinals, ROLLUP and grouping sets remain unsupported. Grouped/aggregate queries referencing sensitive-name source columns fail closed even when renamed. Existing budgets and disclosure checks remain; small groups are not an anonymity guarantee. For example, `SELECT status, COUNT(*) AS measured FROM public.orders GROUP BY status` profiles a grouped relation without fetching its rows.

Query profiles also record `has_unmodeled_expressions`: a source-dependent non-column projection or any aggregate (including `COUNT(*)`) sets it to `true`. Profiling remains available, but `infer-spec` rejects such profiles rather than silently generating independent values that break SQL-expression dependencies. Direct column aliases remain eligible. Older SQL profiles missing the marker must be reprofiled before automatic inference. Non-query legacy profiles retain prior inference behavior. The optional marker defaults to null (unknown); unknown does not change legacy profile fingerprints, while explicit true/false is included in the fingerprint. This does not translate SQL into formulas or authorize raw query-result access.

The profile records `source_fingerprint` and `source_policy_version`, not SQL text or literals. The adapter performs a no-row schema probe followed by bounded aggregates. Query result rows are never returned, persisted, sent to a provider or MCP, or supplied to generation.

DatasetSpec

`DatasetSpec` is the reviewed, executable generation contract. It contains:

- entities, fields, types, nullability, and row counts;
- primary keys and relationships;
- distributions and constraints;
- privacy annotations;
- structured business rules;
- generation and validation settings and schema version.

Infer a spec:

```
test-data-agent infer-spec out/profile.json \
  --count 100 \
  --output out/dataset_spec.yaml
```

Review the spec before generation. Inferred relationships and constraints are candidates, not unquestionable facts.

Generation

Generation reads the specification, not source rows:

```
test-data-agent generate out/dataset_spec.yaml \
  --seed 12345 \
  --format csv \
  --output out/generated
```

The seed belongs to the generation request and is recorded in the effective spec and manifest.

Validation

```
test-data-agent validate out/dataset_spec.yaml out/generated \
  --output out/generated/validation_report.json
```

Validation is deterministic Python code. An AI client may plan or summarize the workflow, but it is not the only validator.

Versioning

Newly constructed `DatasetSpec` files use `schema_version: "1.1"`; version `1.0` remains supported, and serialized files without a version load as `1.0` for compatibility. Exact DECIMAL fields require `1.1`. Keep the schema version, package version, `spec_sha256`, `business_validation.rules_sha256`, and seed with any dataset that must be reproduced later.

The complete field reference is in DatasetSpec.

Safety Model

The central invariant is:

> Generated output must contain newly generated values, not copied source rows.

Source data may be read to derive bounded metadata. It is not used as a pool of rows to shuffle, duplicate, or export.

Trust Boundaries

Selective transformation is a separate mixed-origin surface in stable `1.6.0`. Its execution and approval contract requires local interactive approval of exact input/review bytes before any explicitly non-sensitive field can be preserved. Agent execution can only consume an existing matching receipt; it cannot issue or broaden approval. Sensitivity conflicts, unknown fields and default DECIMAL preservation remain blocked. Receipt integrity does not prove personhood or anonymity. Existing generation, profiling and default responses retain their source-free contracts. Private implementation review does not activate public registration.

Treat all of these as untrusted:

- CSV, JSON, YAML, and Parquet files;
- paths supplied to CLI or MCP tools;
- profile payloads supplied by another process;
- business-rule files and inline rule payloads;
- Trino identifiers, metadata, query results, and environment variables;
- PostgreSQL and Trino JDBC-style endpoint strings;
- local PostgreSQL and Trino SQL query-source files;
- output directories that may contain symlinks or existing files.

Defenses

Sensitive data

Likely PII and secrets are detected from both field names and values. Sensitive columns suppress raw top values and expose masked patterns or aggregate metadata instead. Sensitive examples use an opaque placeholder rather than intentionally preserving source values. Non-sensitive category profiles replace source values with ranked synthetic labels by default. An explicit field-scoped local

allowlist may preserve only a reviewed bounded non-sensitive business enum or constant after content, cardinality, and value-length checks. Rare free text, PII, secrets, identifiers, and quasi-identifiers are never eligible.

Approved exact values may appear only in local profiles, reviewed specs, deterministic generated rows, and local SQL export. External providers receive field-scoped labels, and default MCP responses, logs, and public errors remain source-literal free.

Source-row reuse

Generation uses a reviewed specification and a local seeded random generator. Runtime checks reject exact source-row reuse in supported CSV workflows. The comparison covers complete rows only and runs only when the source CSV is available to that workflow. It does not detect every partial-value, quasi-identifier, or statistical similarity between source and output.

Filesystem

Input and output paths must be distinct. New bundles are assembled in a temporary sibling directory and published only after size and validation checks. Workspace-bound MCP paths reject traversal and symlink escapes.

Trino

Trino access is read-only, allowlisted, bounded by client row limits and server-side time and scan budgets. Unsafe query shapes and likely sensitive projections are rejected before execution. The default aggregate-only tools return only source-literal-free metadata and aggregates. The explicit opt-in row-returning tool `run_safe_select` recursively masks every returned string in bounded scalar or composite values and any non-string field or value recognized as sensitive. Other non-string source values may remain, so those rows are not source-free or anonymous.

PostgreSQL

Direct PostgreSQL profiling uses the optional driver through a forced read-only session. Schema, table, and column allowlists are mandatory, and one shared statement/result/deadline budget bounds metadata and aggregate queries. Normal table profiling accepts no caller SQL and reads no source rows. `profile-query` is a separate local-file surface for one strictly parsed, single-table, fully allowlisted

`SELECT`. Adapter-owned no-row schema and aggregate queries profile its virtual relation without fetching result rows. The profile stores a SHA-256 fingerprint and policy version, never query text, query literals, backend messages, endpoints, or source rows. Qualified local category selectors may retain only values that pass the selective local policy above.

PostgreSQL `schema.table.*` and Trino `catalog.schema.table.*` column selectors are authorization syntax, not SQL. They are expanded from bounded metadata for one exact allowed table into a deterministic explicit-column snapshot before aggregates run. They do not authorize source rows, local exact values, category literals, caller projection stars, providers, default MCP literals, logs, or errors.

PostgreSQL and Trino may accept credential-free JDBC-style endpoint syntax. Parsing happens before client construction and rejects userinfo, secrets, session-changing or unknown properties, malformed escapes, ambiguous paths, and conflicting component configuration. The complete URL is discarded after normalization and is never written to profiles, manifests, MCP responses, provider payloads, logs, or public errors.

Resource limits

The project limits input bytes, rows, columns, cells, YAML complexity, Parquet expansion, generated rows, output bytes, available disk reserve, rule work, query work, and wall-clock generation time.

What The Project Guarantees

For supported synthetic-generation workflows and accepted inputs, the project is designed to:

- avoid copying complete source rows;
- avoid writing raw detected PII into safe profiles;
- regenerate identifiers and sensitive semantic values;
- produce logically deterministic output for the same spec, rules, seed, locale, runtime, dependencies, serializer, and generator version;
- validate schema, relationships, constraints, and business rules;
- record provenance and safety flags in a generation manifest.

The manifest records those effective inputs plus SHA-256 evidence for generated artifacts. Byte-for-byte identity is expected only when the complete recorded environment is unchanged. It is not guaranteed across Python, dependency,

serializer, operating-system, or package versions, even when logical values and validation results remain equivalent.

What It Does Not Guarantee

The project cannot decide whether every ambiguous business field is sensitive. It also cannot prove that an inferred distribution is legally or statistically safe for every use case. The project does not currently implement differential privacy, k-anonymity, l-diversity, or a privacy budget, and makes no statistical privacy guarantee. Treat generated data as requiring a domain-specific privacy review before distribution.

Human review is still required for:

- domain-specific identifiers and rare free text;
- inferred relationships and constraints;
- privacy requirements outside the built-in detectors;
- organization-specific retention and access controls;
- downstream systems that may log or redistribute generated data.

Do not send production data, raw PII, credentials, or tokens to an LLM while reviewing profiles or specifications.

Acceptance Checklist

Before publishing a generated dataset:

This checklist applies to synthetic generation, not the gated mixed-origin transformation workflow. A transformed bundle must not be relabelled synthetic merely because validation passed: its action provenance and any matching local preservation receipt remain separate requirements.

- manifest says `synthetic: true`;
- manifest says `source_rows_copied: false`;
- deterministic validation passed;
- business validation passed when rules were used;
- seed, row counts, format, and fingerprints are correct;
- profile and generated samples contain no unexpected sensitive values;
- the output is stored outside the source path.

See Review The Output for the practical review sequence.

Configured SQL Transformation Sessions

The separately selected configured query workflow captures one allowlisted read-only result using administrator configuration, then reviews and validates frozen local bytes without reconnecting. It does not expand ordinary profiling or source-free generation authority. MCP returns only metadata and consumes an existing exact local controlling-TTY receipt when preservation requires one; it cannot issue approval. Session handles expire without recapture, owned raw inputs/receipts are removed, and only validated selected output is retained. Cleanup failure refuses further work. Registration is not data access permission or final exact-SHA security/release clearance.

Help

Troubleshooting

Start with:

```
test-data-agent doctor
test-data-agent doctor --json
```

The final line should be `doctor passed`.

Require the optional capability you intend to operate:

```
test-data-agent doctor --require-extra parquet
```

For Parquet this performs a local temporary generation and read-back, checks row counts and manifest safety flags, and contacts no external service. A missing extra recommends installation. An installed extra whose local smoke fails reports a capability failure instead of recommending reinstall; check local setup and rerun doctor. A broken dependency import is reported as an import failure, not as an absent package. Exception text and temporary paths are omitted. If quickstart generation fails, earlier dependency checks remain in the report and `doctor --json` returns `ok: false`, exit code 1, and a failed quickstart check without exposing the underlying exception.

`doctor --require-extra mcp` constructs the real generator `FastMCP` transport, registers one local audited probe tool, and verifies its public tool listing. It does not start a server, listen on a port, invoke the tool, or contact an MCP client.

`doctor --require-extra trino` validates a bounded allowlisted query with the installed Trino SQL parser, constructs a client for the reserved `doctor.invalid` host, and closes it without opening a cursor or executing SQL. It does not read Trino credentials or contact a coordinator. A missing extra gets installation guidance; a local smoke failure does not establish a missing package or remote connectivity problem.

`doctor --require-extra openai` constructs and closes the installed SDK client with a local non-secret placeholder and verifies the structured Responses API used by the advisor adapter. It does not read `OPENAI_API_KEY`, send a request, or contact the provider. A local smoke failure does not establish a missing package or a problem with deployment credentials.

`doctor --require-extra gigachat` uses a local fake SDK client to verify strict structured-response mapping and cleanup. It does not read `GIGACHAT_CREDENTIALS` or `GIGACHAT_ACCESS_TOKEN`, obtain a token, or contact GigaChat. Install stable `1.6.0` with the `gigachat` extra. See release status.

JDBC-Style URL Rejected

JDBC-style endpoint input, qualified column wildcards, and `profile-query` are available in stable `1.5.0` and retained in stable `1.6.0`.

Use only the documented credential-free PostgreSQL or Trino shape. Keep users, password references, tokens, roles, headers, proxies, session properties, allowlists, and budgets in their existing settings. Remove duplicate or unknown URL properties. When URL and component fields are both set, make the explicit host, port, database/TLS or catalog/schema values identical, or remove one representation. Errors intentionally omit the URL and conflicting values.

An endpoint also fails when the URL exceeds 4,096 UTF-8 bytes, its host exceeds 253 characters, a PostgreSQL database identifier exceeds 63 characters, or a Trino catalog/schema identifier exceeds 255 characters. Shorten the local configuration; the rejected value is intentionally absent from the error.

Qualified Column Wildcard Rejected

Use exactly `schema.table.*` for PostgreSQL or `catalog.schema.table.*` for Trino. Keep the parent schema/table or catalog/schema allowlists present. Trino wildcards require restricted mode. Bare `*`, schema-wide forms, embedded stars, empty or

duplicated metadata, schema drift, and expansion beyond the configured column budget fail before aggregate profiling. Replace the wildcard with reviewed exact columns when the table is wider than the intended profiling scope.

SQL Query Source Rejected

Pass one UTF-8 file to `profile-query`; SQL text is not accepted in an option or environment variable. SQL source policy 1.1 permits one fully qualified, single-table `SELECT` with explicit output aliases, bounded filters, and the documented deterministic scalar subset. Remove joins, CTEs, subqueries, set or window operations, table functions, comments, volatile/unknown functions, multiple statements, and unauthorized tables or columns.

Aliased SUM/COUNT/MIN/MAX/AVG and explicit-source-column GROUP BY are allowed. Other selected columns must be grouping keys. Remove nested/wrapped aggregates, DISTINCT aggregates, HAVING, grouping expressions/ordinals and grouping sets. Sensitive-name source references remain blocked for grouped/aggregate queries, even with renamed outputs. An accepted aggregate profile is not permission to fetch result rows or automatically infer a dependency-preserving generation spec.

The query file, AST, projected fields, and adapter work must remain inside all configured budgets. A changed file, unsupported type, malformed backend metadata, schema drift, or budget exhaustion fails closed and leaves no profile. Errors intentionally omit SQL text, literals, endpoints, and backend messages; inspect the local reviewed file and allowlists instead.

Transformation Rejected

Stable `1.6.0` provides the common-profile CLI/MCP workflow. See the CLI workflow. Published `1.5.0` does not include this command. Registration grants no dataset permission. The finite fictional transformation/session campaign is recorded in stable evidence; it grants no blanket dataset consent.

If the snapshot no longer matches, review the saved inputs, policies and mappings again; validate and execute with the newly returned digest. A receipt for an earlier snapshot cannot authorize the changed one. When preservation is selected, obtain a matching receipt through the separate local terminal workflow; an agent or MCP client cannot issue one.

For a budget refusal, inspect the declared input, review and output limits before selecting another bounded run. Do not interpret partial work or elapsed process time as successful acceptance. If cleanup is incomplete, inspect the reported retained artifacts before retrying into a new destination. Existing files are not permission to overwrite or resume a failed publication.

GigaChat Advice Failed

Check the fixed local category first: missing extra, authentication, scope, TLS/CA bundle, rate limit, timeout, filtered response, invalid response, or unavailable service. Remote response bodies and SDK exception text are intentionally suppressed because they may reflect credentials or request metadata.

Configure exactly one of `GIGACHAT_CREDENTIALS` and `GIGACHAT_ACCESS_TOKEN`. Match `GIGACHAT_SCOPE` to the API project, keep TLS verification enabled, and use `GIGACHAT_CA_BUNDLE_FILE` only for a reviewed readable CA bundle. Do not work around a certificate failure with an insecure SDK example. A failed provider call leaves the workspace awaiting review and does not create generated rows; retry only after correcting the local cause.

GigaChat's structured-output feature is currently beta. The adapter performs a single local baseline-compatibility pass only after initial structured validation fails, then reruns every normal validation. If `invalid response` persists, the remaining provider proposal is not safe to review; use another supported model or retry later instead of weakening validation.

Command Not Found

Symptom:

```
test-data-agent: command not found
```

Activate the environment where the package was installed:

```
source .venv/bin/activate
python3 -m pip show agent-paranoid-android
```

On Windows PowerShell:

```
.venv\Scripts\Activate.ps1
python -m pip show agent-paranoid-android
```

Output Already Exists

Folder workflows without overwrite support require a new or empty directory. For example, choose a new path for `generate-from-example`:

```
test-data-agent generate-from-example data/example_dataset \
  --count 25 \
  --seed 12345 \
  --format csv \
  --output out/run-002
```

DatasetSpec `generate --overwrite` replaces the entire output folder, including obsolete files, after generation and validation. Review the destination first; see folder replacement and rollback.

Single-entity file bundles use separate ownership checks. A different primary filename or format, invalid or missing manifest, stale sidecar, or unrelated sibling is rejected before replacement. Use a new output folder instead of deleting evidence from an incomplete or mixed file bundle. Never point output at a source file or source folder.

Missing Optional Dependency

Optional commands print a version-pinned installation command and exit `69`. Run that command in the same environment as `test-data-agent`, then verify the capability locally:

```
python -m pip install "agent-paranoid-android[postgres]==PACKAGE_VERSION"
test-data-agent doctor --require-extra postgres
```

Replace the extra and version with the exact values from the error. A passing local doctor check does not verify credentials or contact a remote service.

Malformed Input Or Unexpected Traceback

Malformed YAML/JSON and invalid models are normal input failures: they exit `2`, publish no success artifact, and do not show a traceback. Use `--json` in automation to branch on `error.code`.

Unexpected internal failures exit `70` with a fixed bounded message. Retry with `--debug` only in a trusted terminal where technical paths and exception context are safe to display. Provider response bodies and credentials remain redacted even in

normal provider errors.

Cancelled Operation

Ctrl+C exits `130` after catchable staging cleanup or bundle rollback. The message confirms that no successful final bundle was published. Inspect an agent workspace with `agent-status`; for a non-agent command, keep any unexpected destination for investigation and retry into a new path.

Agent Approval Was Interrupted

Inspect the workspace:

```
test-data-agent agent-status out/agent
```

When it reports `recovery_required`, run the exact `agent-recover` command it prints. Recovery requires the previously reviewed DatasetSpec SHA-256, revalidates the existing generated bundle, and does not generate new rows.

Do not edit files under `generated/` before recovery. A changed checkpoint, manifest, report, spec, profile, or row file causes recovery to fail closed.

Process Or Host Stopped During Publication

For an agent workspace, run `agent-status` first. Use `agent-recover` only when the status reports `recovery_required`; it revalidates the existing generated bundle before publishing missing completion metadata.

For a non-agent generation command, do not treat a hidden staging directory or a destination without its expected manifest and validation report as success. Confirm that no generation process is still running, retain the incomplete files for investigation when needed, and rerun with the same reviewed inputs and seed into a new destination.

Where an atomic state writer or staged bundle publication is used, replacement prevents readers from observing its partial state during normal operation. Standalone artifact commands are not one global transaction. Shared atomic single-file writes and owned directory publication use `fsync`, but complete multi-file

crash consistency is not guaranteed. A hard process stop, host or storage failure, or power loss can still leave staging data or an incomplete bundle. Use storage with the durability and backup guarantees required by the deployment.

Input Limit Exceeded

The error names the failed limit. Prefer splitting an oversized source or reducing requested rows before raising the corresponding environment variable.

When a limit must change, set the smallest value that supports the reviewed workload and keep output and wall-clock limits in proportion.

See Configuration.

Sensitive Value Rejected

Profiles and business rules reject values that resemble PII, credentials, tokens, or private keys.

Do not bypass the detector by encoding or fragmenting a production value. Replace it with a semantic rule, a reserved example value, or a generator strategy.

Validation Failed

Open:

- `validation_report.json`;
- `business_validation_report.json`, when present;
- the effective spec and rule file.

Check the first failing section before changing generation settings. Common causes are an incorrect inferred relationship, impossible field ranges, conflicting formulas, and a business rule that references the wrong field.

Negative and mixed modes can fail validation intentionally. Keep their output separate and label it as invalid test data.

Results Are Not Reproducible

Confirm that both runs use the same:

- package version;
- `DatasetSpec` and its fingerprint;
- business-rule file and fingerprint;
- seed;
- row count, mode, invalid ratio, and format.

File ordering and output encoding should also be compared on the same supported platform.

Trino Allowlists Are Required

Set both variables:

```
export TRINO_ALLOWED_CATALOGS=hive,iceberg
export TRINO_ALLOWED_SCHEMAS=test_data,staging
```

Do not use `TRINO_ALLOW_UNRESTRICTED=true` merely to silence configuration errors.

Plain HTTP Is Disabled

Use HTTPS for remote Trino. For an isolated local integration instance only:

```
export TRINO_HTTP_SCHEME=http
export TRINO_ALLOW_INSECURE_HTTP=true
```

MCP Path Rejected

Move the input or output below `TEST_DATA_AGENT_WORKSPACE_ROOT`. A textual path that appears to be inside the workspace can still be rejected when an existing symlink resolves outside it.

Use a real directory with no symlink boundary and request a new output path.

Reporting A Security Problem

Do not open a public issue containing exploit details, source data, PII, or credentials. Follow the repository security policy.

Known Issues

Stable `1.6.0` completed public verification. External acceptance feedback was reconciled against current main on 2026-10-22; the feedback work plan separates historical fixes, current limitations and unverified observations.

- CSV numeric strings can remain phone-like and sensitive. Do not bypass privacy checks or switch exact measures to float to suppress an error; reviewed numeric profiling is planned. Exact decimal generation keeps its existing reviewed `--decimal-fields` contract.
- Trino already supports explicit runtime authentication over verified HTTPS. The external authentication failure still needs configuration/version evidence; fake-driver tests do not establish acceptance against a particular server.
- Numeric shape supports optional zero/subunit frequencies and a minimum subunit exponent. Older profiles can omit that evidence; reprofile before assuming it exists. A negative exponent is valid for values below one. Parquet distinct counts can be unavailable; missing statistics do not mean zero distinct values.
- Relationship validation checks declared relationships. Missing logical edges, source orphan rates and temporal granularity are not established by a passing validation report. Use reviewed declarations and explicit entity row counts.

These are development follow-ups, not changes already published in `1.6.0`.

The separate transformation/session acceptance campaign remains open, including genuine local-terminal preservation acceptance where required. Existing automated and historical RC results do not establish its completion or private-data suitability. Provider adapters remain experimental; resource and artifact durability limits remain in their respective guides.

Resolved findings and their original risk dispositions are retained in Historical Security Findings.